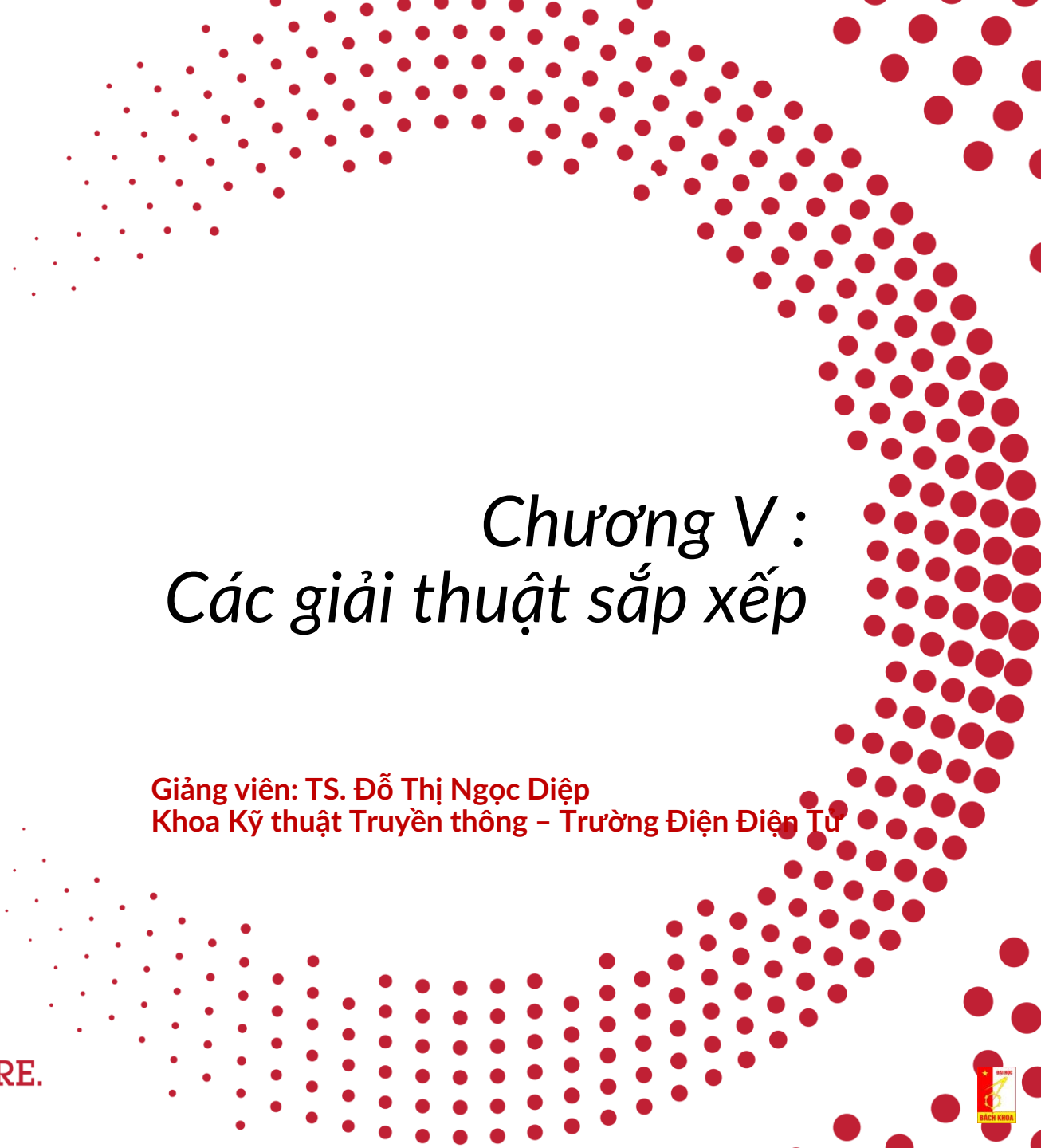


# HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT ET2100

ONE LOVE. ONE FUTURE.



# Chương V : *Các giải thuật sắp xếp*

Giảng viên: TS. Đỗ Thị Ngọc Diệp  
Khoa Kỹ thuật Truyền thông – Trường Điện Điện Tử

ONE LOVE. ONE FUTURE.

1. Đặt vấn đề
2. Các giải thuật sắp xếp cơ bản
  - Sắp xếp chọn (Selection sort)
  - Sắp xếp nổi bọt (Bubble/Exchange sort)
  - Sắp xếp chèn (Insertion sort)
3. Các giải thuật sắp xếp nâng cao
  - Sắp xếp nhanh/phân đoạn (Quick sort)
  - Sắp xếp vun đống (Heap sort)
  - Sắp xếp trộn (Merge sort)

# 1. Đặt vấn đề

- **Bài toán tổng quát:** Cho trước một dãy  $N$  phần tử  $a_1, a_2, \dots, a_N$ . Cần tìm giải thuật sắp xếp các phần tử của dãy trên theo một thứ tự (tiêu chuẩn) nào đó.
  - Mục đích: Hỗ trợ các bài toán hiển thị, tìm kiếm, v.v.
  - Ứng dụng trong thực tế:
    - Từ trong từ điển
    - Files trong một thư mục
    - Chỉ mục trong sách
    - Lịch sự kiện trên báo
    - Danh sách các khóa học sắp xếp theo Khoa, Mã HP
    - Catalog trong thư viện
    - V.v.

# 1. Đặt vấn đề

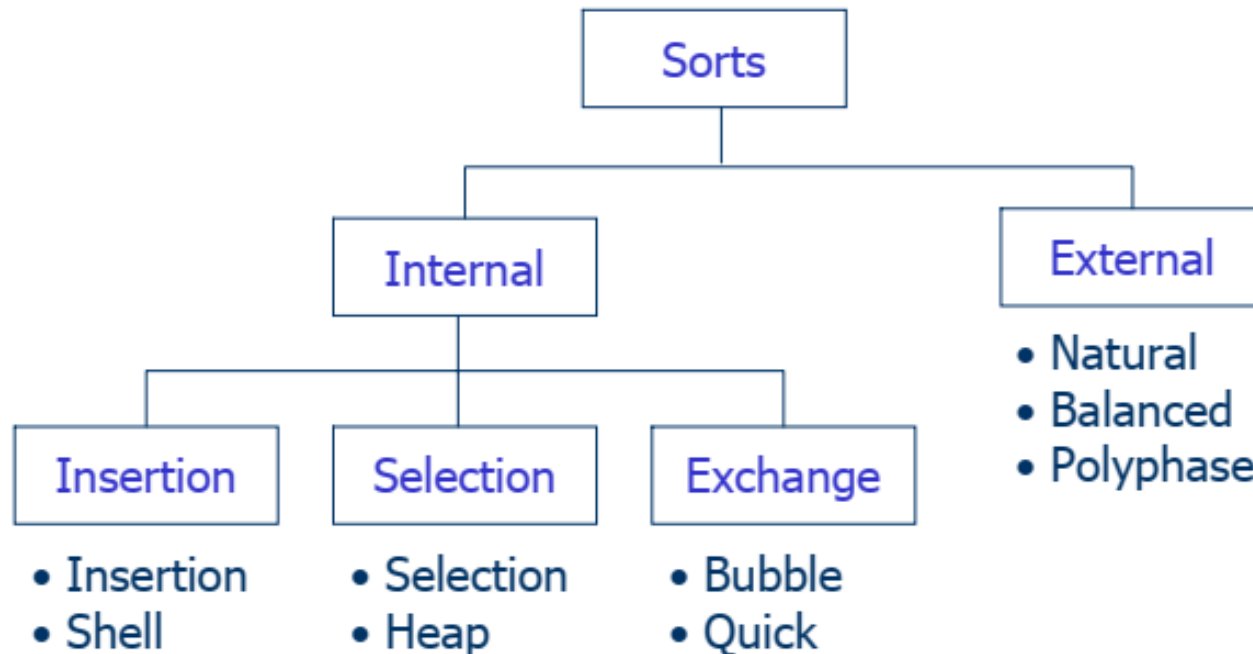
- **Bài toán** : Không giảm tính tổng quát của các giải thuật sắp xếp, đồng thời để đơn giản hóa việc trình bày → minh họa các giải thuật thông qua việc **sắp xếp một dãy  $N$  số theo trật tự tăng dần**.
- Với mỗi giải thuật, cần đưa ra:
  - Ý tưởng giải thuật
  - Cài đặt cơ bản (gồm 1 hoặc 1 số hàm)
  - Đánh giá giải thuật

# 1. Đặt vấn đề

- Phân loại:
  - **Giải thuật sắp xếp cơ bản:**
    - Ở mỗi bước, đưa từng phần tử của dãy cần SX vào đúng vị trí của nó trong dãy kết quả
    - Không cần quan tâm đến vị trí của các phần tử khác
  - **Giải thuật sắp xếp nâng cao:**
    - Ở mỗi bước, đưa từng phần tử vào đúng vị trí của nó trong dãy kết quả, đồng thời kết hợp bố trí hợp lý các phần tử còn lại, nhằm giảm thiểu số thao tác ở những bước sau
  - Các Giải thuật sắp xếp nâng cao thường chạy nhanh hơn các Giải thuật sắp xếp cơ bản, nhưng cũng thường phức tạp hơn trong ý tưởng giải thuật và cài đặt.

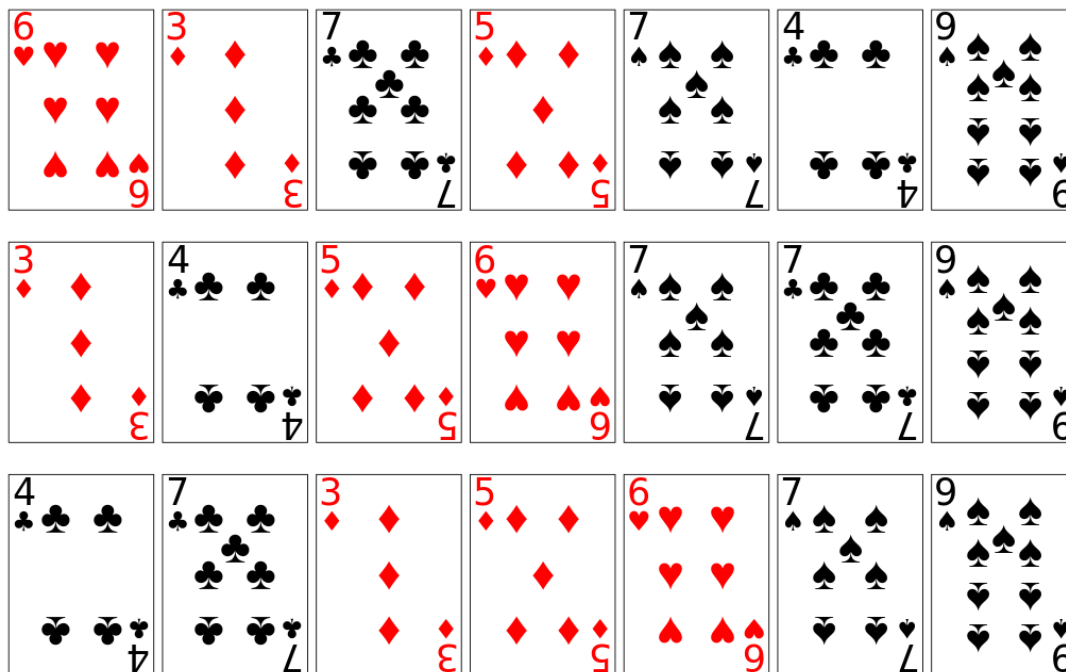
# 1. Đặt vấn đề

- Phân loại:
  - **Sắp xếp trong** (Internal sorting): Sắp xếp dữ liệu lưu trữ trong bộ nhớ máy tính
  - **Sắp xếp ngoài** (External sorting): Sắp xếp dữ liệu lưu trữ trong file, khi lượng dữ liệu lớn, không thể lưu trữ trong bộ nhớ trong



# 1. Đặt vấn đề

- Phân loại:
  - **Sắp xếp nhiều khóa:** Trong ứng dụng thực tế, dựa trên một khóa không đủ để phân biệt dữ liệu/bản ghi -> cần sử dụng thông tin từ nhiều khóa
  - Có thể sắp xếp dựa trên khóa chính, rồi đến khóa phụ 1, 2, v.v.



## 2. Các giải thuật sắp xếp cơ bản

- Sắp xếp chọn
- Sắp xếp nổi bọt
- Sắp xếp chèn

## 2.1 Sắp xếp chọn (*Selection sort*)

- Ý tưởng giải thuật
  - Đầu vào: dãy  $N$  số  $a_1, a_2, \dots, a_N$
  - Đầu ra: dãy vào đã được sx theo chiều tăng dần
  - Giải thuật:
    - GT thực hiện trong  $N-1$  bước, đánh số các bước  $i=1, 2, \dots, N-1$
    - Ở bước thứ  $i$ , tìm số nhỏ thứ  $i$  rồi đưa nó vào vị trí thứ  $i$  trong dãy.
- Sau lượt thứ  $i$ , dãy cần sắp coi như được chia thành 2 phần:
  - Phần đã sắp: từ vị trí 1 đến  $i$
  - Phần chưa sắp: từ vị trí  $i+1$  đến  $n$
  - Ở bước thứ  $i$ , chọn phần tử nhỏ nhất trong số các phần tử chưa được sắp, đưa phần tử được chọn vào vị trí đúng bằng phép đổi chỗ

## 2.1 Sắp xếp chọn

- Minh họa: Cho dãy số với  $N = 7$

|         |   |   |   |   |          |   |          |
|---------|---|---|---|---|----------|---|----------|
|         | 3 | 2 | 4 | 5 | 1        | 7 | 6        |
| $i = 1$ | 1 | 2 | 4 | 5 | <u>3</u> | 7 | 6        |
| $i = 2$ | 1 | 2 | 4 | 5 | 3        | 7 | 6        |
| $i = 3$ | 1 | 2 | 3 | 5 | <u>4</u> | 7 | 6        |
| $i = 4$ | 1 | 2 | 3 | 4 | <u>5</u> | 7 | 6        |
| $i = 5$ | 1 | 2 | 3 | 4 | 5        | 7 | 6        |
| $i = 6$ | 1 | 2 | 3 | 4 | 5        | 6 | <u>7</u> |

## 2.1 Sắp xếp chọn

- Giải thuật

```
for (i=1; i<=N-1; i++) {  
    Tìm m sao cho  $a_m = \text{Min}(a_i, a_{i+1}, \dots, a_N)$  ;  
    Hoán đổi  $a_i$  và  $a_m$  ;  
}
```

- Cài đặt:

- Cài đặt dãy số: Mảng, Danh sách
- Kiểu dữ liệu của phần tử: kiểu đơn giản, kiểu có cấu trúc
  - Kiểu có cấu trúc: cần xây dựng quy tắc so sánh kiểu dữ liệu
- Tùy bài toán, Lựa chọn các kiểu cài đặt phù hợp

- Cài đặt cơ bản

```
typedef int DaySo[N];

void SXChon (DaySo a) {
    int i, j, m;
    for (i=0; i<N-1; i++) {
        //Tìm m: am=Min(ai, ai+1, ..., aN-1)
        m = i;
        for(j=i+1; j<N; j++)
            if (a[j] < a[m]) m = j;
        //Đưa am về đầu dãy
        if (m != i) HoanDoi(a[i], a[m]);
    }
}
```

## 2.1 Sắp xếp chọn

- Cài đặt cơ bản
  - C/C++

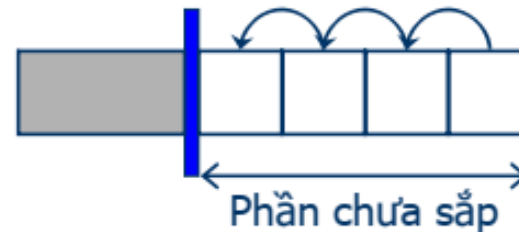
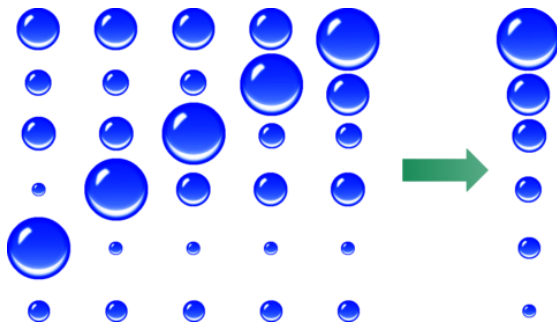
```
void selectionSort(int A[], int N) {  
    int m;  
    for (int i=0; i < N-1; i++){  
        m=i;  
        for (int k=i+1; k < N; k++)  
            if (A[k] < A[m]) m=k;  
        if (m != i) swap(A[i], A[m]);  
    }  
}
```

## 2.1 Sắp xếp chọn

- Đánh giá giải thuật
  - Thời gian thực hiện thuật toán:
    - Trường hợp tốt nhất:  $O(n^2)$ 
      - Dãy ban đầu đã được sắp xếp
      - 0 phép đổi chỗ, thực hiện  $n(n-1)/2$  phép so sánh
    - Trường hợp xấu nhất:  $O(n^2)$ 
      - $n-1$  phép đổi chỗ,  $n(n-1)/2$  phép so sánh
  - Độ phức tạp thời gian trung bình  $O(n^2)$
  - Nhận xét:
    - Đơn giản, dễ cài đặt
    - Được sử dụng với dữ liệu nhỏ
    - Không hiệu quả khi sử dụng với dữ liệu lớn

## 2.2. Sắp xếp nổi bọt (Bubble/Exchange sort)

- Ý tưởng giải thuật:
  - Đầu vào: dãy N số  $a_1, a_2, \dots, a_N$
  - Đầu ra: dãy vào đã được sx theo chiều tăng dần
  - Ý tưởng:
    - Dãy cần sắp được chia thành 2 phần: phần đã sắp, phần chưa sắp
    - Thông qua phép đổi chỗ, tại mỗi lượt phần tử nhỏ nhất trong phần chưa được sắp sẽ được “đẩy dần” lên trước và cuối cùng nhập vào phần được sắp.

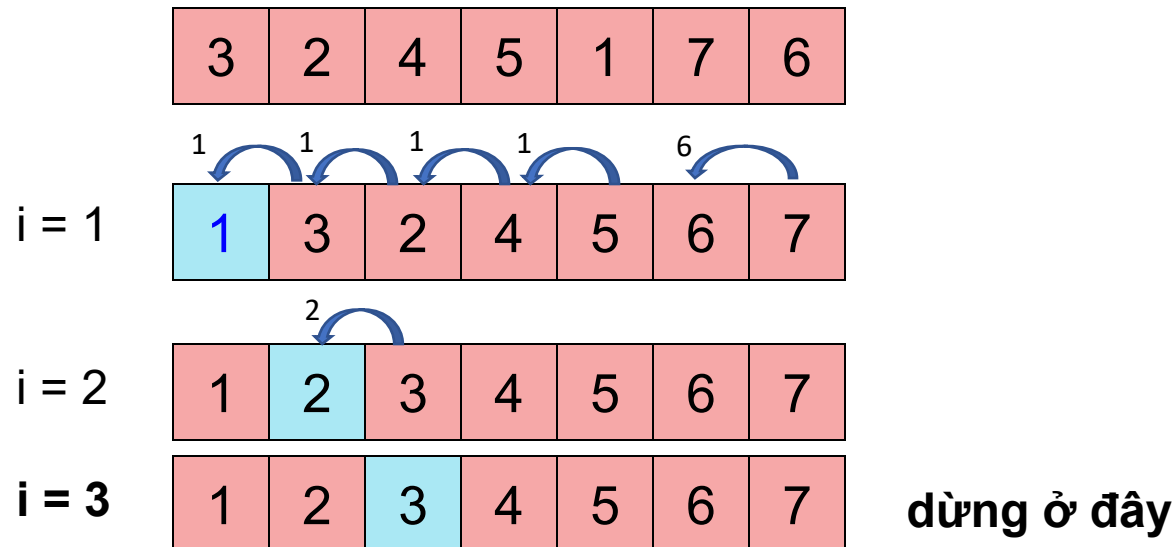


## 2.2. Sắp xếp nổi bọt

- Ý tưởng giải thuật:

- Giải thuật:

- GT thực hiện trong **tối đa** là  $N-1$  bước, đánh số các bước  $i=1,2,3,\dots$
- Ở bước thứ  $i$ , so sánh lần lượt  $N-i$  cặp số ( $a_k$  và  $a_{k+1}$ ), với  $k=N-1, N-2, \dots, i$ ; sao cho nếu  $a_k > a_{k+1}$  thì Hoán Đổi  $a_k$  và  $a_{k+1}$
- Nếu ở một bước  $i$  nào đó mà ta không phải hoán đổi cặp nào thì chứng tỏ dãy đã sắp xếp, nên dừng GT ở đây.



## 2.2. Sắp xếp nổi bọt

- Giải thuật thô
  - Không kiểm tra điều kiện dừng

```
void SXNoiBot (DaySo a) {  
    for (i = 1; i < N; i++) {  
        for (k=N-1; k>=i; k--)  
            if ( $a_k > a_{k+1}$ ) {  
                HoanDoi ( $a_k$ ,  $a_{k+1}$ );  
            }  
    }  
}
```

## 2.2. Sắp xếp nổi bọt

- Giải thuật cải tiến
  - Có kiểm tra điều kiện dừng

```
void SXNoiBot (DaySo a) {  
    i = 1;  
    sorted = False;  
    while (!sorted && i<N) {  
        sorted = True;  
        for (k=N-1; k>=i; k--)  
            if (ak > ak+1) {  
                HoanDoi (ak, ak+1);  
                sorted = False;  
            }  
        i++;  
    }  
}
```

## 2.2. Sắp xếp nổi bọt

- Cài đặt giải thuật

```
void bubbleSort(int A[], int N) {  
    int i = 0, k;  
    int sorted = 0;  
    while (!sorted && i < N-1) {  
        sorted = 1;  
        for (k=N-2; k>=i; k--)  
            if (A[k] > A[k+1]) {  
                swap(A[k], A[k+1]);  
                sorted = 0;  
            }  
        i++;  
    }  
}
```

## 2.2. Sắp xếp nổi bọt

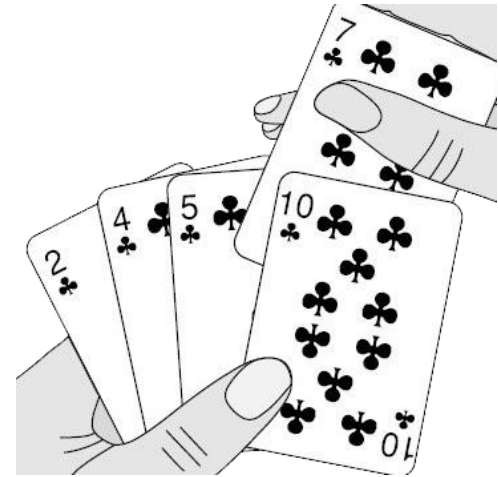
- Đánh giá giải thuật
  - Thời gian thực hiện thuật toán:
    - Trường hợp tốt nhất:  $O(n)$ 
      - Dãy ban đầu đã được sắp xếp, giải thuật chỉ cần đúng 1 bước.
      - 0 phép đổi chỗ,  $(n-1)$  phép so sánh (+ phép gán)
    - Trường hợp xấu nhất:  $O(n^2)$ 
      - Phải thực hiện  $n-1$  bước
      - Ở bước thứ  $i$ , thực hiện  $n-i$  phép so sánh và đổi chỗ (+ phép gán)
  - Độ phức tạp thời gian trung bình  $O(n^2)$
  - Nhận xét:
    - Đơn giản, dễ cài đặt
    - Được sử dụng với dữ liệu nhỏ, gần như đã sắp xếp
    - Không tốt hơn giải thuật sắp xếp chọn, mặc dù trong trường hợp tốt nhất nó nhanh hơn rất nhiều.

## 2.3 Sắp xếp chèn (*Insertion sort*)

- Ý tưởng giải thuật:

- Xây dựng dãy được sắp xếp có số phần tử tăng dần, ban đầu là 1 phần tử, sau đó là 2 phần tử, ... và cuối cùng đủ  $N$  phần tử được sắp xếp.
- Ở mỗi bước, ta cần chèn một phần tử trong dãy chưa được sắp xếp vào dãy các phần tử đã được sắp xếp, sao cho sau khi chèn thì dãy mới cũng được sắp xếp.

- Đầu vào: dãy  $N$  số  $a_1, a_2, \dots, a_N$
- Đầu ra: dãy vào đã được sx theo chiều tăng dần
- Giải thuật:
  - GT thực hiện trong  $N-1$  bước, đánh số các bước  $i=1, 2, \dots, N-1$
  - Ở bước thứ  $i$ , chèn phần tử  $a_{i+1}$  vào dãy  $(a_1, a_2, \dots, a_i)$  đã được sắp xếp trước đó, sao cho sau khi chèn ta được dãy mới cũng được sắp xếp.



## 2.3 Sắp xếp chèn

- Giải thuật chèn

- Đầu vào:

- Dãy  $(a_1, a_2, \dots, a_i)$  đã được sắp xếp, và phần tử cần chèn  $b$

- Đầu ra:

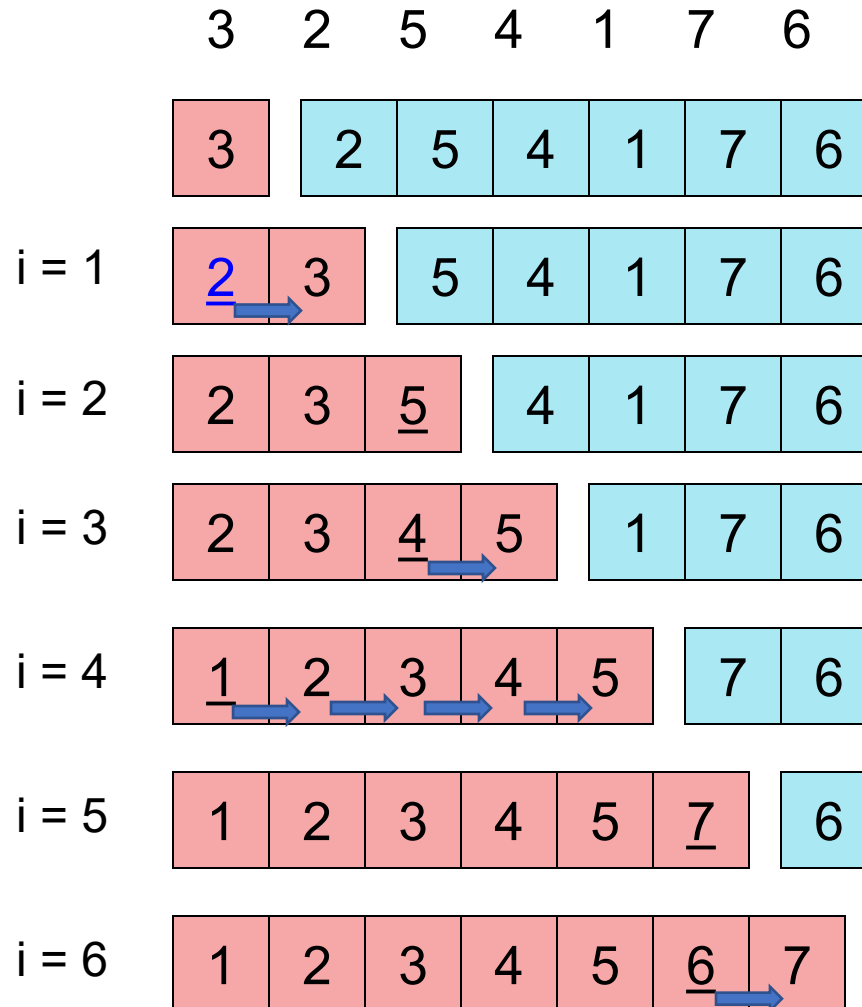
- Dãy  $(a_1, a_2, \dots, a_i, a_{i+1})$  cũng được sắp xếp

- Giải thuật:

- Tìm vị trí  $k$  cần chèn trong dãy ban đầu thỏa mãn các điều kiện sau:
      - $1 \leq k \leq i+1$  và  $a_{k-1} \leq b < a_k$
    - Dịch các phần tử  $(a_k, a_{k+1}, \dots, a_i)$  sang phải một vị trí để nhường chỗ cho phần tử  $b$
    - Chèn  $b$  vào vị trí  $k$
    - Lưu ý: để kết hợp việc tìm vị trí  $k$  và dịch các phần tử sang phải  $\rightarrow$  tiến hành tìm từ cuối dãy trở về đầu

## 2.3 Sắp xếp chèn

- Minh họa hoạt động của GT



- Giải thuật

```
void SXChen (DaySo a) {  
    int i, j;  
    int k;  
    for (i=1; i<N; i++) {  
        k = i;  
        b = ai+1;  
        while (k>=1 AND b<ak) {  
            ak+1 = ak;  
            k--;  
        }  
        ak+1 = b;  
    }  
}
```

- Cài đặt

```
void insertionSort(int A[], int N) {  
    int i,k,b;  
    for (i=0;i < N-1;i++){  
        k = i;  
        b = A[i+1];  
        while (k>=0 && b<A[k]){  
            A[k+1] = A[k];  
            k--;  
        }  
        A[k+1] = b;  
    }  
}
```

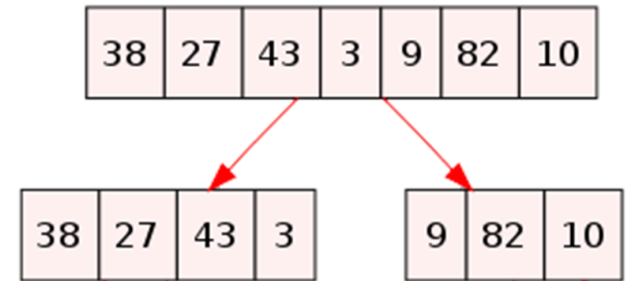
## 2.3 Sắp xếp chèn

- Đánh giá giải thuật
  - Thời gian thực hiện thuật toán:
    - Trường hợp tốt nhất:  $O(n)$ 
      - Dãy ban đầu đã được sắp xếp, giải thuật vẫn thực hiện trong  $n-1$  bước.
      - Mỗi bước, 0 phép đổi chỗ, 1 phép so sánh (+ phép gán)
    - Trường hợp xấu nhất:  $O(n^2)$ 
      - Phải thực hiện  $n-1$  bước (dãy ban đầu có thứ tự ngược lại)
      - Ở bước thứ  $i$ , thực hiện  $i$  phép so sánh và  $i$  phép dịch phần tử (+ phép gán)
  - Độ phức tạp thời gian trung bình  $O(n^2)$
- Nhận xét:
  - Dễ triển khai, hiệu quả trên tập nhỏ dữ liệu, dữ liệu đã sắp xếp một phần
  - Thực thi tốt hơn sắp xếp chọn (x1.4) và sắp xếp nổi bọt (x2)
  - Có thể thực thi online, vừa sắp xếp vừa thêm phần tử mới

### 3. Các giải thuật sắp xếp nâng cao

### 3. Các giải thuật sắp xếp nâng cao

- Các giải thuật sắp xếp cơ bản có độ phức tạp  $O(n^2)$ 
  - Thời gian tính toán tăng nhanh khi  $n$  lớn
  - -> Chia nhỏ bài toán thành các bài toán nhỏ hơn
- Hướng tiếp cận chia để trị (triển khai đệ quy)
  - Chia thành hai phần
  - Sắp xếp từng phần riêng biệt
  - Hợp nhất hai phần đã sắp xếp
- Các giải thuật sắp xếp nâng cao
  - Sắp xếp nhanh (Quick Sort)
  - Sắp xếp trộn (Merge Sort)
  - Sắp xếp vun đống (Heap Sort)



## 3.1 Sắp xếp nhanh/phân đoạn (QuickSort)

- Ý tưởng giải thuật:

- Sử dụng 1 giá trị **c** gọi là chốt: pivot
- Bố trí các phần tử của dãy sao cho tất cả các phần tử đứng trước chốt đều nhỏ hơn c, và tất cả các phần tử đứng sau chốt đều lớn hơn hoặc bằng c.
- Lặp lại cách làm này với các tập nhỏ hơn bên trái và bên phải của chốt

- Trường hợp 

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 2 | 1 | 3 | 5 | 6 | 9 | 7 | 8 |
|---|---|---|---|---|---|---|---|

ng
-   
 $< pivot$        $pivot$        $\geq pivot$

- Sử dụng quá trình **Phân đoạn** (partition), nên giải thuật còn được gọi là **Sắp xếp Phân đoạn**

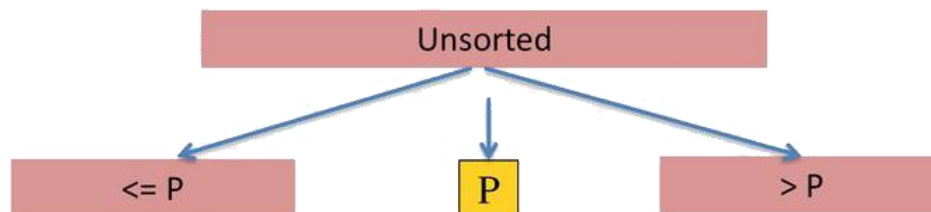
- Kết thúc việc phân đoạn, kết hợp các tập nhỏ đã sắp xếp lại → tập ban đầu đã sắp xếp

## 3.1 Sắp xếp nhanh/phân đoạn

- Ý tưởng giải thuật:

- ...

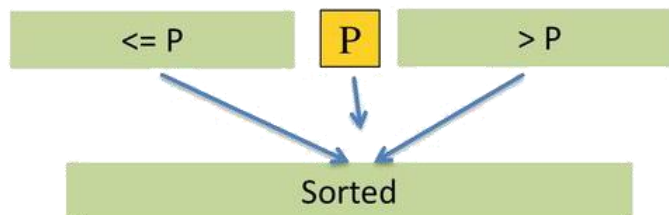
**Divide:** Pick a pivot, partition into groups



**Conquer:** Return array when length  $\leq 1$

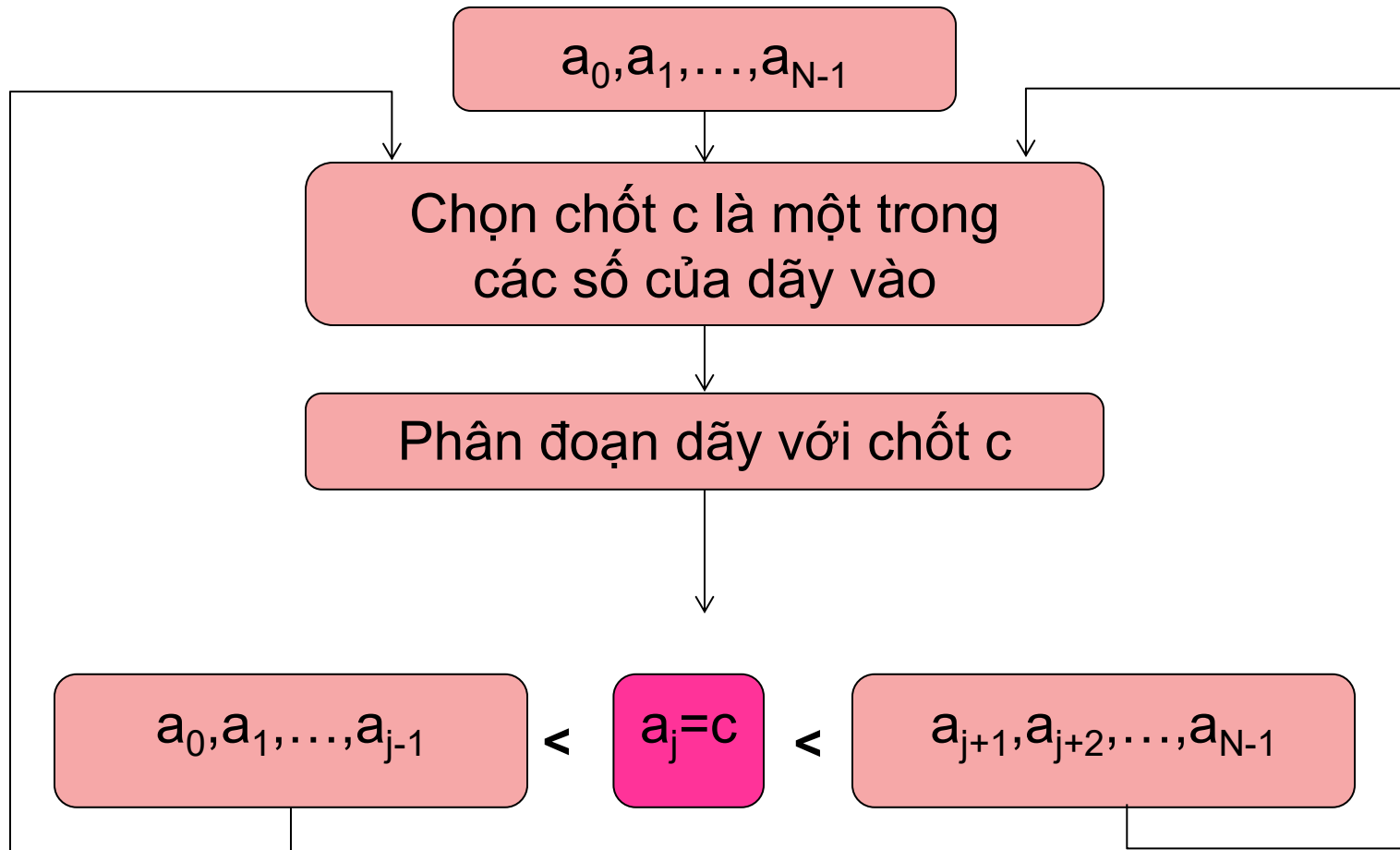


**Combine:** Combine sorted partitions and pivot



## 3.1 Sắp xếp nhanh/phân đoạn

- Mô tả ý tưởng giải thuật



## 3.1 Sắp xếp nhanh/phân đoạn

- Mô tả ý tưởng giải thuật

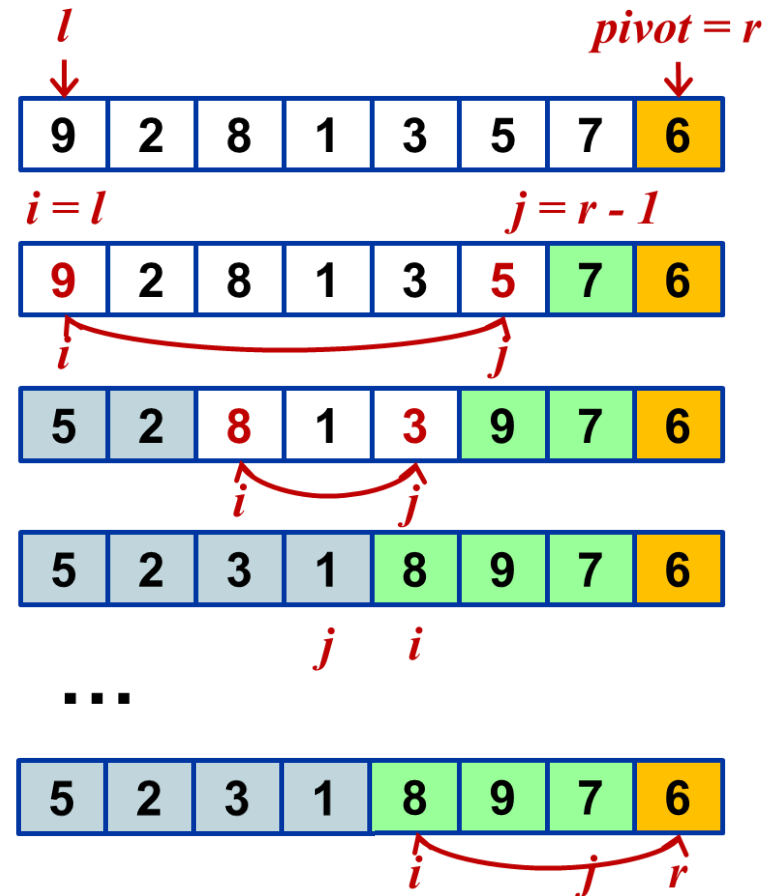
```
void sortQuickRange(DaySo a, int l, int r) {  
    if (l < r) {  
        i = qsPartitionXXX(a, l, r);  
        sortQuickRange(a, l, i - 1);  
        sortQuickRange(a, i + 1, r);  
    }  
}  
  
void sortQuick(DaySo a) {  
    sortQuickRange(a, 0, a.length - 1);  
}
```

## 3.1 Sắp xếp nhanh/phân đoạn

- Giải thuật phân đoạn **qsPartitionXXX**
  - Lựa chọn chốt
  - Đưa chốt về vị trí đúng trong dãy
- Có thể có nhiều biến thể :
  - Cách lựa chọn chốt:
    - Chọn chốt là phần tử đứng đầu hoặc cuối dãy
    - Chọn phần tử đứng giữa dãy làm chốt
    - Chọn phần tử trung vị trong 3 phần tử đứng đầu, đứng giữa và đứng cuối dãy
    - Chọn phần tử bé nhất (giải thuật sắp xếp chọn)
    - Chọn phần tử ngẫu nhiên
    - v.v
  - Cách duyệt phần tử

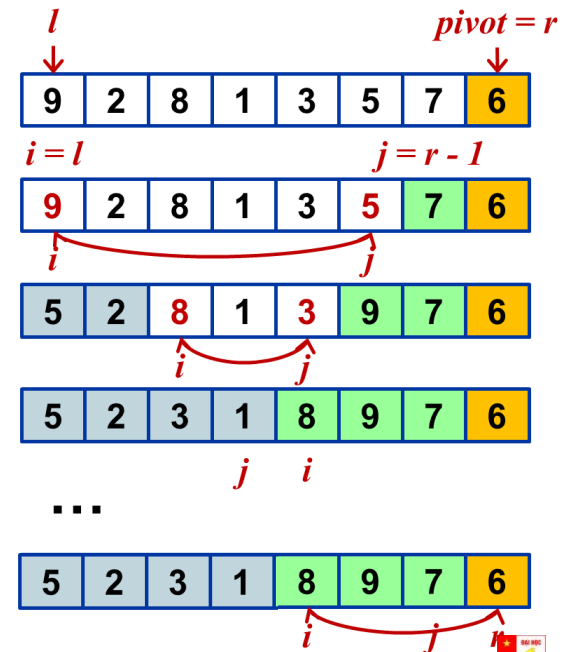
- Giải thuật phân đoạn Hoare (Hoare Partition)

- Chọn phần tử cuối dãy là chốt
- $l, r$  : chỉ số đầu, cuối của dãy
- $i$  chạy xuôi từ đầu dãy  
 $j$  chạy ngược từ cuối dãy
- Repeat until  $i \geq j$ 
  - $i$ : vị trí phần tử đầu tiên  $\geq$  pivot
  - $j$ : vị trí phần tử đầu tiên  $<$  pivot
  - Đổi chỗ phần tử tại  $i$  và  $j$
- Đổi chỗ phần tử tại  $i$  và  $r$



- Giải thuật phân đoạn Hoare (Hoare Partition)

```
int PartitionHoare(DaySo a, int l, int r) {  
    int pivot = a[r];  
    int i, j;  
  
    for (i = l, j = r - 1; i < j;) {  
        for (; a[i] < pivot && i < j; i++);  
        for (; a[j] >= pivot && i < j; j--);  
        HoanDoi(a[i], a[j]);  
    }  
  
    if (a[r] > a[i]) return r;  
    HoanDoi(a[i], a[r]);  
    return i;  
}
```



- Lựa chọn phần tử đầu dãy làm chốt: => Tự viết

## 3.1 Sắp xếp nhanh/phân đoạn

- Đánh giá giải thuật
  - Trường hợp xấu nhất:  $O(n^2)$
  - Trường hợp tốt nhất:  $O(n \log n)$
  - Trường hợp trung bình:  $O(n \log n)$
- Nhận xét:
  - Nhanh hơn các giải thuật cơ bản
  - Có thể áp dụng trên các tập dữ liệu kích thước nhỏ, trung bình, lớn
  - Giải thuật phức tạp, có tính đệ quy
  - Phương án Lựa chọn chốt:
    - Bất kỳ phần tử nào cũng có thể được chọn làm chốt (không cần phải là phần tử cuối cùng / đầu tiên)
    - Chọn chốt là phần tử đầu tiên hoặc cuối cùng có thể dẫn đến trường hợp xấu nhất khi mảng đã được sắp xếp

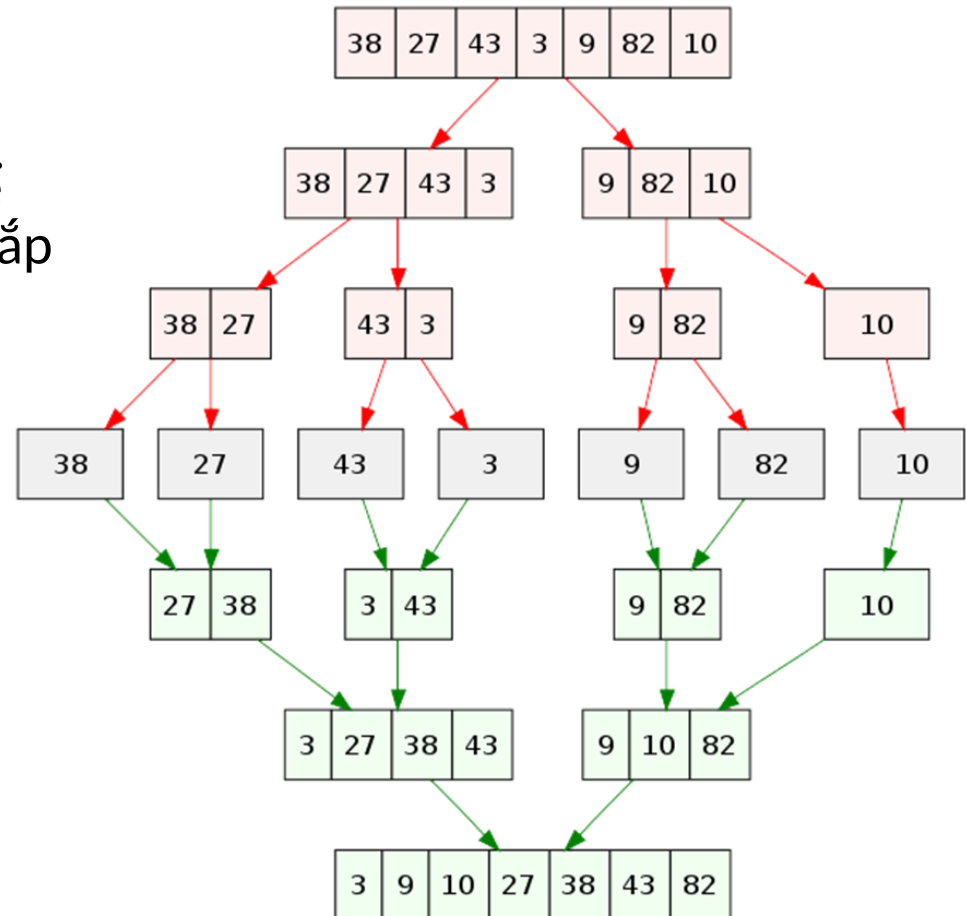
## 3.2 Sắp xếp trộn (Merge Sort)

- Ý tưởng giải thuật:

- Chia dãy ban đầu thành hai dãy con có kích thước khác nhau không quá 1
- Sắp xếp hai dãy con trên
- Trộn hai dãy con đã sắp xếp để được dãy ban đầu cũng được sắp xếp
- **Điểm dừng:** khi kích thước của dãy  $\leq 1$

- 2 thao tác chính

- Chia dãy
- Trộn dãy kèm sắp xếp



## 3.2 Sắp xếp trộn

- Cài đặt giải thuật:

- Chia dãy ban đầu thành hai dãy con có kích thước khác nhau không quá 1
- [Sắp xếp hai dãy con trên]
- Trộn hai dãy con đã sắp xếp để được dãy mới cũng được sắp xếp

```
void sortMergePart(DaySo a, int L, int R) {  
    int m = floor((L + R) / 2);  
    if (L < m) sortMergePart(a, L, m);  
    if (m + 1 < R) sortMergePart(a, m + 1, R);  
    mergePartsXXX(a, L, m, R);  
}
```

**Điểm dừng ?**

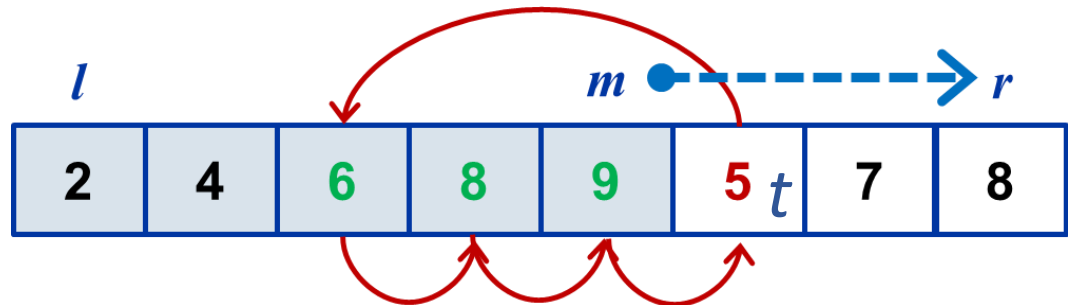
```
void sortMerge(DaySo a, int n) {  
    sortMergePart(a, 0, n-1);  
}
```

- Đầu vào:
  - 2 dãy  $p=(p_0,p_1,...,p_{m-1})$  và  $q=(q_0,q_1,...,q_{n-1})$  đã được sắp xếp tăng dần
- Đầu ra:
  - Dãy  $a=(a_0,a_1,...,a_{m+n-1})$  là kết quả trộn của  $p$  và  $q$  và cũng được sắp xếp
- Giải thuật:
  - Khởi tạo: dùng 2 biến chạy  $i=j=0$ ;  $a =$  rỗng
  - Chừng nào ( $i < m$  và  $j < n$ ) // 2 dãy  $p$  và  $q$  còn chưa được chạy hết
    - Nếu  $p_i \leq q_j$  thì  $\{ a = a + p_i ; i++ ; \}$
    - Trái lại, thì  $\{ a = a + q_j ; j++ ; \}$
  - Nếu  $i \geq m$  (dãy  $p$  đã được chạy hết) thì  $a = a + (q_j, q_{j+1}, ..., q_{n-1})$
  - Trái lại thì  $a = a + (p_i, p_{i+1}, ..., p_{m-1})$

```
void mergeParts_aux(DaySo a, int l, int m, int r) {
    int pn = m - l + 1; //số pt mảng trái
    int qn = r - m;      //số pt mảng phải
    int p[pn] = a[l]..a[l+pn];
    int q[qn] = a[m + 1]..a[m + 1 + qn];
    // i: duyệt mảng p, j: duyệt mảng q
    for (i = 0, j = 0; i < pn || j < qn; ) {
        if ((i == pn) || (j < qn && p[i] > q[j])) {
            a[l + i + j] = q[j];
            j++;
        } else {
            a[l + i + j] = p[i];
            i++;
        }
    }
}
```

- Do 2 mảng p và q là liên tục, phép trộn thực hiện ngay trên mảng đầu vào a, dựa trên việc so sánh và đẩy lùi phần tử về phía sau

```
void mergeParts_inPlace(DaySo a, int l, int m, int r) {  
    int i, t;  
    for (; m < r; m++) {  
        t = a[m+1];  
        for (i = m; i >= l && a[i] > t; i--)  
            a[i+1] = a[i];  
        a[i+1] = t;  
    }  
}
```



## 3.2 Sắp xếp trộn

- Đánh giá giải thuật

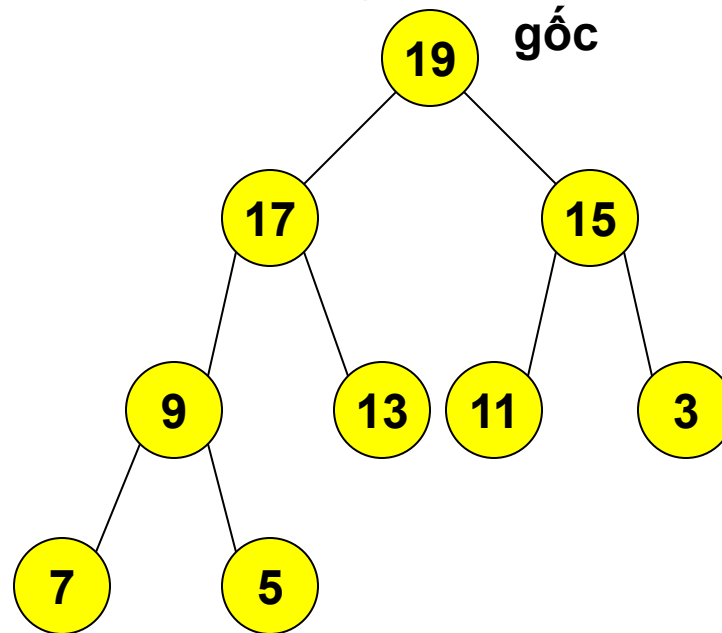
| Complexity         | Standard method                 | In-place method                   |
|--------------------|---------------------------------|-----------------------------------|
| <b>Computation</b> | <b><math>O(n \log n)</math></b> | <b><math>O(n^2 \log n)</math></b> |
| - Division         | $O(\log n)$                     | $O(\log n)$                       |
| - Merging          | $O(n)$                          | $O(n^2)$                          |
| <b>Memory</b>      | <b><math>O(n)</math></b>        | <b><math>O(1)</math></b>          |

- Nhận xét:

- Giải thuật trộn sử dụng bộ nhớ tạm dùng thêm bộ nhớ  $O_{\text{memory}}(n)$  nhưng có thời gian tính toán ít hơn giải thuật trộn tại chỗ  $O(n \log_2 n)$
- Giải thuật trộn tại chỗ không dùng thêm bộ nhớ  $O_{\text{memory}}(1)$  nhưng có thời gian tính toán cao hơn giải thuật sử dụng bộ nhớ tạm  $O(n^2 \log^2 n)$

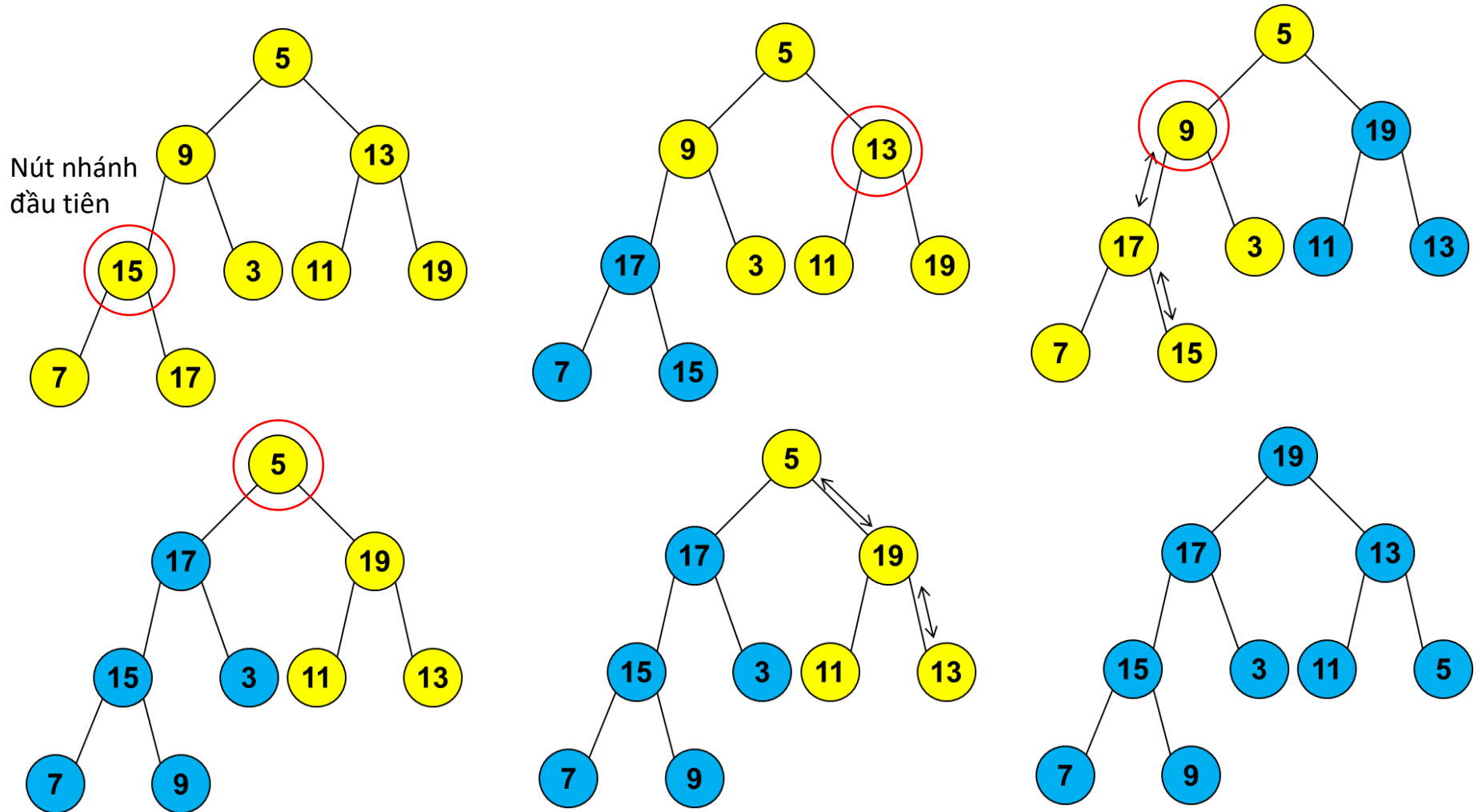
### 3.3 Sắp xếp vun đống (*Heap Sort*)

- Một số khái niệm cơ bản
  - **Đống** (Heap) hay *cây thứ tự bộ phận*: Là một cây nhị phân hoàn chỉnh mà *mọi cây con* của nó (bao gồm cả chính cây đó) có gốc là nút có giá trị lớn nhất
  - **Vun đống**: là quá trình đưa một cây hoàn chỉnh mà chưa phải là đống chuyển thành một đống



- Đưa một cây nhị phân hoàn chỉnh có  $N$  nút bất kỳ thành một đồng.
  - Thực hiện vun dần từ dưới lên trên.
  - Do các nút lá đã là đồng, nên việc vun đồng *bắt đầu từ các nút nhánh*, theo thứ tự lần lượt từ các nút nhánh ngoài cùng cho đến các nút nhánh ở trong và cuối cùng cho đến gốc (các nút nhánh có thứ tự giảm dần).
  - Mỗi lần vun đồng cho nút nhánh phải đảm bảo cây con của nó cũng là một đồng

- Minh họa Vun đồng 1 cây có N nút

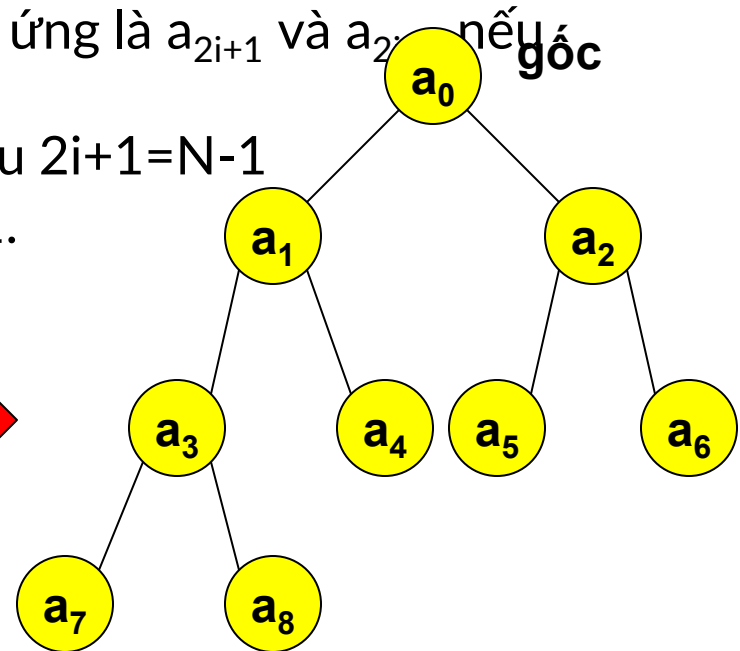


### 3.3 Sắp xếp vun đống

- Dãy cần sắp xếp được biểu diễn thành cây nhị phân hoàn chỉnh.
- Sắp xếp dãy  $\Leftrightarrow$  Lặp lại việc vun đống cây nhị phân
- Cài đặt cây nhị phân hoàn chỉnh bằng cấu trúc mảng:
  - Mỗi phần tử của mảng là một nút của cây
  - Phần tử  $a_0$  chính là nút gốc của cây
  - Phần tử  $a_i$  sẽ có hai con trái, phải tương ứng là  $a_{2i+1}$  và  $a_{2i+2}$  nếu  $2i+2 \leq N-1$

hoặc chỉ có một con trái, nếu  $2i+1 = N-1$   
hoặc là nút lá nếu  $2i+1 > N-1$ .

$a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$

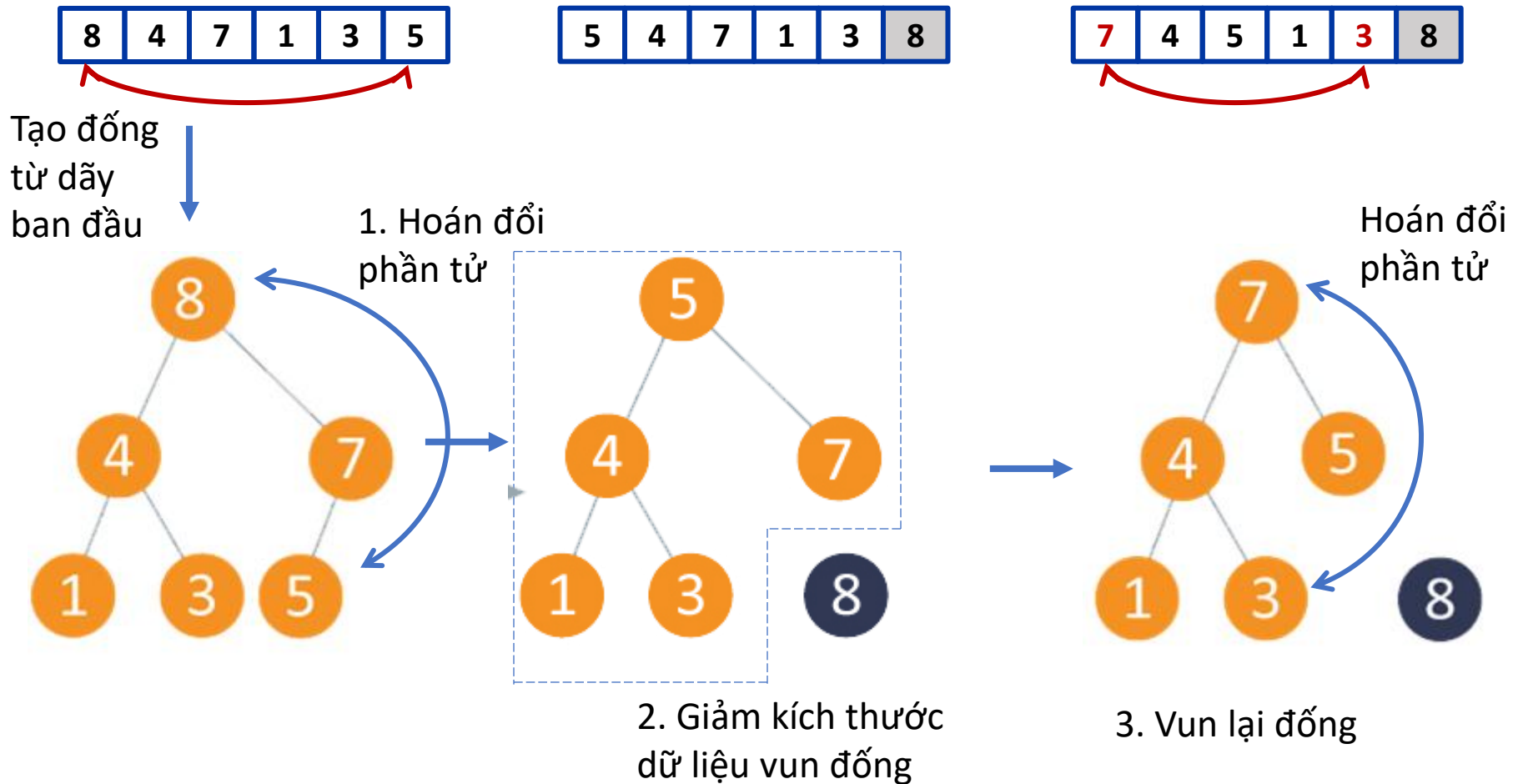


### 3.3 Sắp xếp vun đồng

- Dãy cần sắp xếp được biểu diễn thành cây nhị phân hoàn chỉnh.
- Sắp xếp dãy  $\Leftrightarrow$  Lặp lại việc Vun đồng cây nhị phân
- **Sắp xếp vun đồng:**
  - B1: Thực hiện Vun đồng cây nhị phân
  - B2: Hoán đổi phần tử đầu tiên (phần tử gốc cây, phần tử max) với phần tử cuối cùng của dãy ( $a_0, a_{N-1}$ )
  - B3: Vun lại đồng bắt đầu từ nút gốc  $a_0$ , kích thước dữ liệu vun đồng giảm đi 1.
  - Lặp lại B2, B3 cho đến khi dãy được sắp xếp (N-1 lần)

### 3.3 Sắp xếp vun đống

- Minh họa Sắp xếp vun đống



- Minh họa Sắp xếp vun đống



### • Giải thuật Vun đống **MakeHeap**

- *Đưa một cây có  $N$  nút bất kỳ thành một đống.*
- Xác định các nút nhánh của cây  $a_i$  (các nút  $a_i$  thỏa mãn  $2i+1 \leq N-1$ )
- Lần lượt *vun đống cho các cây con* có nút gốc là các nút nhánh  $a_i$
- Thực hiện theo thứ tự từ ngoài cùng vào

```
void MakeHeap (DaySo A, int N) {  
    if (N<2) return;  
    int m=N/2-1; //Vị trí nút nhánh ngoài cùng  
    for (int i=m; i>=0; i--)  
        MakeSubHeap (A, i, N);  
}
```

*=> Vun đống cây con như thế nào ?*

### 3.3 Sắp xếp vun đống

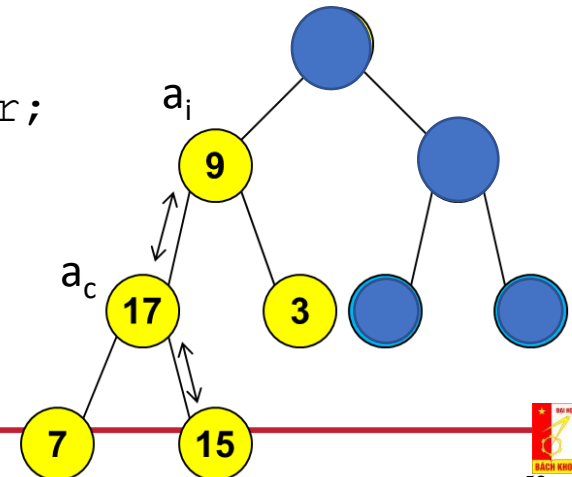
- **Giải thuật Vun đống cho cây con MakeSubHeap**

- có nút gốc là  $a_i$
- các cây con của cây con này cũng đã là các đống

- **Giải thuật:**

- Nếu  $a_i$  đã là nút lá của cây ban đầu -> dừng
- Tìm nút con  $a_c = \text{MAX}(a_{2i+1}, a_{2i+2})$  (nút con có giá trị lớn nhất trong 2 nút con)
  - Hoán đổi ( $a_i, a_c$ )
  - Vun lại cây con có nút gốc là  $a_c$

```
void MakeSubHeap(DaySo A, int i, int N) {  
    int l = 2*i+1, r = 2*i+2, c = l;  
    if (l >= N) return;  
    if (r < N && A[r] > A[l]) c = r;  
    if (A[i] < A[c]) {  
        HoanDoi(A[i], A[c]);  
        MakeSubHeap(A, c, N);  
    }  
}
```



- **Giải thuật Sắp xếp vun đống HeapSort:**

- B1: Thực hiện Vun đống cây nhị phân
- B2: Hoán đổi phần tử đầu tiên (phần tử gốc cây, phần tử max) với phần tử cuối cùng của dãy ( $a_0, a_{N-i}$ )
- B3: Vun lại đống bắt đầu từ nút gốc  $a_0$ , kích thước dữ liệu vun đống giảm đi 1.
- Lặp lại B2, B3 cho đến khi dãy được sắp xếp (N-1 lần)

```
void HeapSort (Dayso A, int N) {  
    if (N<2) return;  
    MakeHeap (A, N) ;  
    for (int i=1; i<N; i++) {  
        HoanDoi (A[0], A[N-i]) ;  
        MakeSubHeap (A, 0, N-i) ;  
    }  
}
```

### 3.3 Sắp xếp vun đống

- Đánh giá giải thuật
  - Vun đống cho một cây con (**MakeSubHeap**) :
    - Trường hợp xấu nhất:  $i=1$ , phải hoán đổi nút gốc theo dọc chiều cao của cây ( $= \lceil \log_2(N + 1) \rceil - 1$ ) :  $O(\log_2 n)$
  - Vun đống (**MakeHeap**) :  $O(n \log_2 n)$
  - Sắp xếp vun đống (**HeapSort**) :  $O(n \log_2 n)$

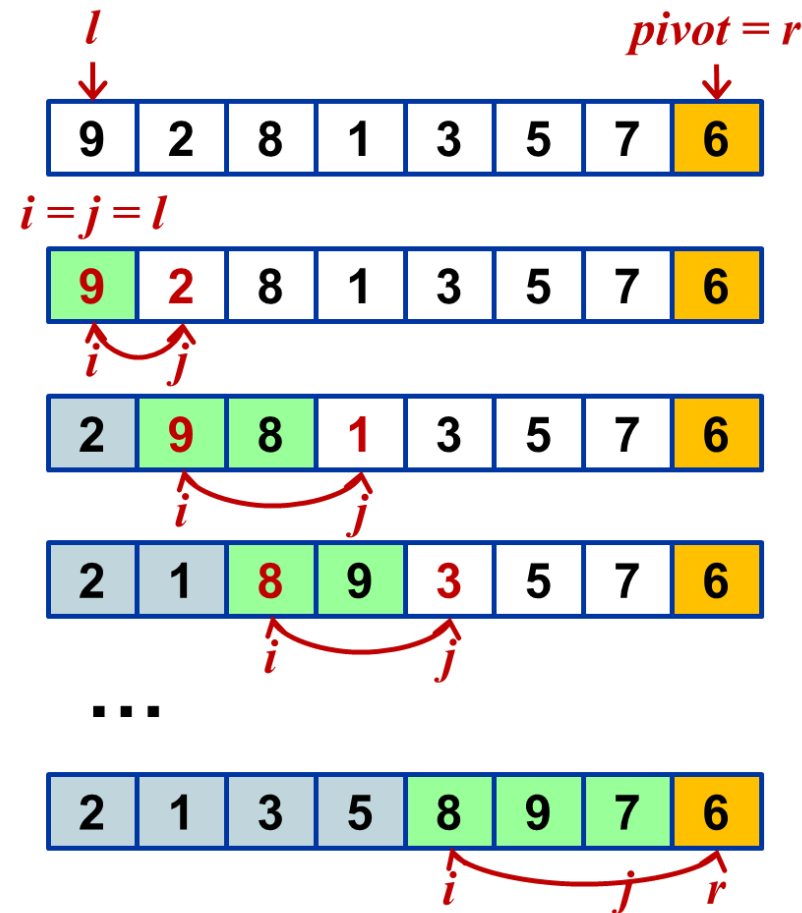
| Algorithm        | Worst case      | Average case    | Memory      |
|------------------|-----------------|-----------------|-------------|
| Bubble           | $O(n^2)$        | $O(n^2)$        | $O(1)$      |
| Insertion        | $O(n^2)$        | $O(n^2)$        | $O(1)$      |
| Selection        | $O(n^2)$        | $O(n^2)$        | $O(1)$      |
| Heap             | $O(n \log n)$   | $O(n \log n)$   | $O(1)$      |
| Merge (std)      | $O(n \log n)$   | $O(n \log n)$   | $O(n)$      |
| Merge (in place) | $O(n^2 \log n)$ | $O(n^2 \log n)$ | $O(1)$      |
| Quicksort        | $O(n^2)$        | $O(n \log n)$   | $O(\log n)$ |

- Bài 1: Cài đặt 3 giải thuật sắp xếp cơ bản cho 1 danh sách được cài đặt bằng móc nối đơn thẳng
- Bài 2: Cài đặt 3 giải thuật sắp xếp nâng cao cho 1 danh sách được cài đặt bằng móc nối đơn (hoặc kép) thẳng





- Giải thuật phân đoạn Lomuto
  - Chọn phần tử cuối dãy là chốt
  - $l, r$  : chỉ số đầu, cuối của dãy
  - $i$ : chỉ số/vị trí của phần tử đầu tiên  $\geq$  pivot;  $\rightarrow$  trước  $i$  là các phần tử nhỏ hơn chốt
  - Duyệt từ đầu đến cuối dãy, nếu có phần tử nhỏ hơn chốt, thì đổi vị trí cho vị trí  $i$ .
    - $j$ : chỉ số duyệt
    - Sau đổi chỗ đánh dấu vị trí mới  $\geq$  pivot:  $i++$
  - Bước cuối cùng, đổi chỗ chốt và phần tử thứ  $i$



## • Giải thuật phân đoạn Lomuto

```
int qsPartitionLomuto(DaySo a, int l, int r) {  
    int pivot = a[r];  
    int i=l, j=l;  
  
    for (i = l, j = l; j < r; j++) {  
        if (a[j] < pivot){  
            HoanDoi(a[i], a[j]);  
            i++;  
        }  
    }  
    HoanDoi(a[i], a[r]);  
    return i;  
}
```

